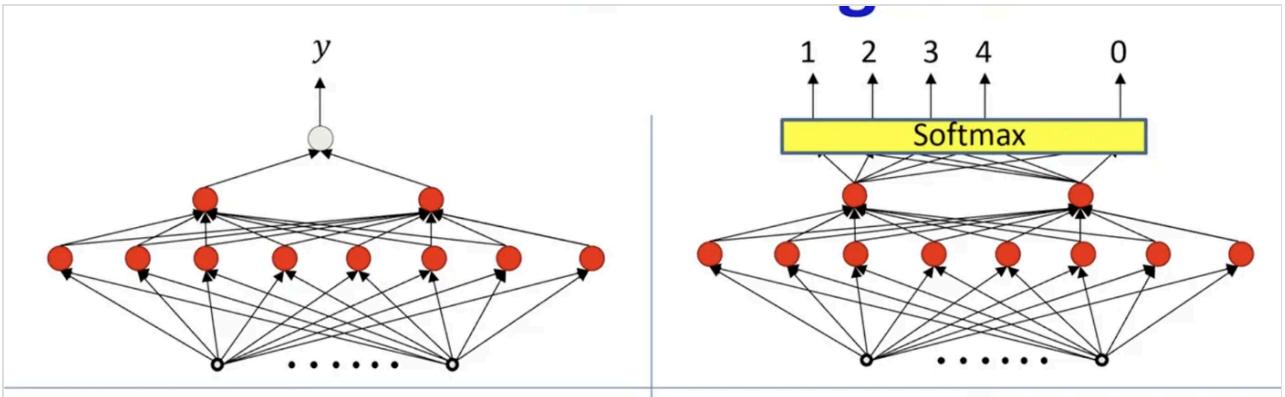
Lec8. Training Part VI

Generalization / Tricks of the trade

Training Neural Nets by Gradient Descent

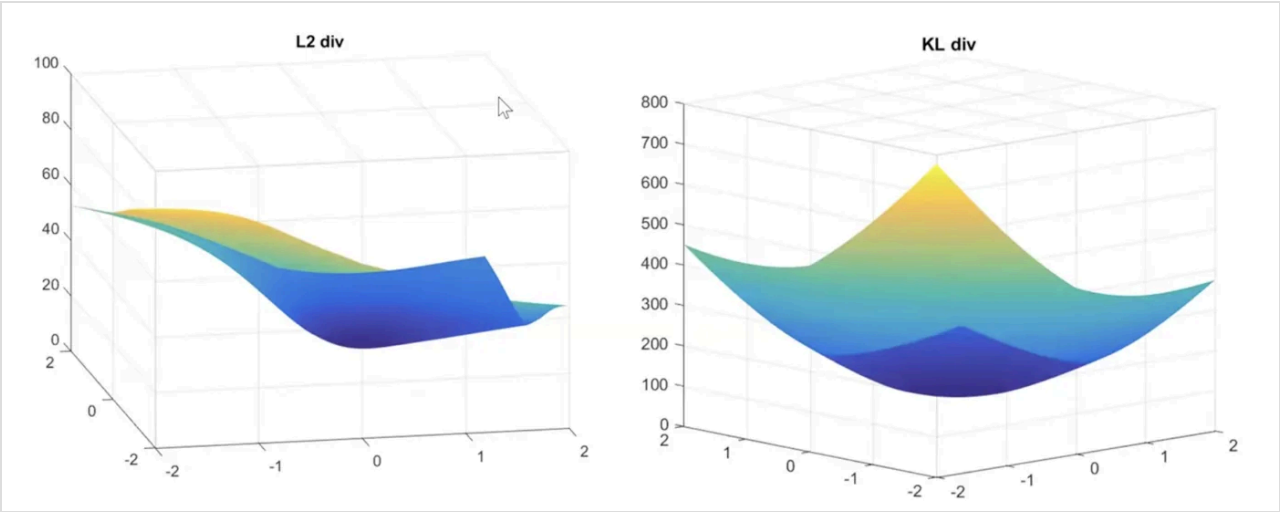
Total training loss:

$$Loss = \frac{1}{T}\sum_t Div(\boldsymbol{Y}_t, \boldsymbol{d}_t; \mathbf{W}_1, \mathbf{W}_2, \ldots, \mathbf{W}_K)$$



Desired output:	d	Desired output: $[0,0,\ldots,1,\ldots,0]$
L2	$Div = \frac{1}{2}(y-d)^2$	$Div = \frac{1}{2}\sum_i (y_i - d_i)^2$
KL	$Div = -d\log(y) - (1-d)\log(1-y)$	$Div = \sum_i d_i \log(d_i) - \sum_i d_i \log(y_i)$

- L2, KL과 같은 divergence를 고를 수 있음.
  ⇒ Loss function을 고르는 느낌.



- KL Div의 경우, 좀 더 학습하기 쉽게(볼록하게) 생김.
  ($w_1, w_2, \ldots$에 관해서 볼 때)
- → activation이 거의 loss function의 surface를 보여줌.

A note on derivatives

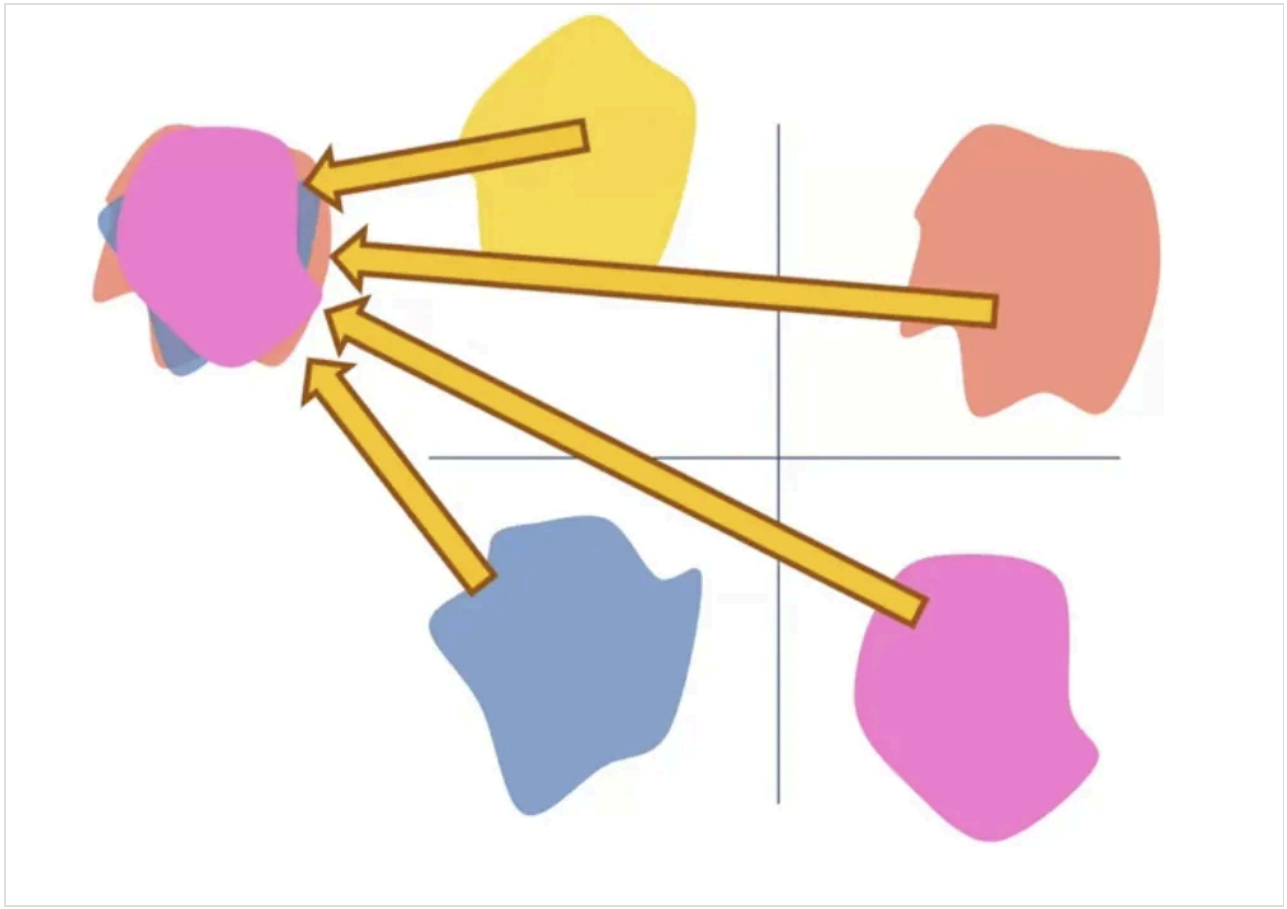
$$\nabla_z \frac{1}{2}\|\boldsymbol{y}-\boldsymbol{d}\|^2 = (\boldsymbol{y}-\boldsymbol{d})^\top$$

$$\nabla_z KL(\boldsymbol{y},\boldsymbol{d}) = (\boldsymbol{y}-\boldsymbol{d})^\top$$

- L2, KL 모두 같은 형태의 $derivative$를 가짐
  → back prop에서 학습하는 건 같게 됨.

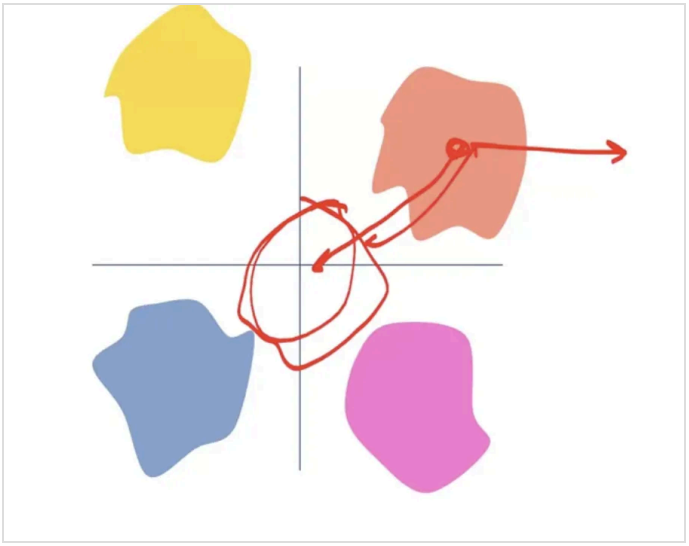
The problem of covariate shifts

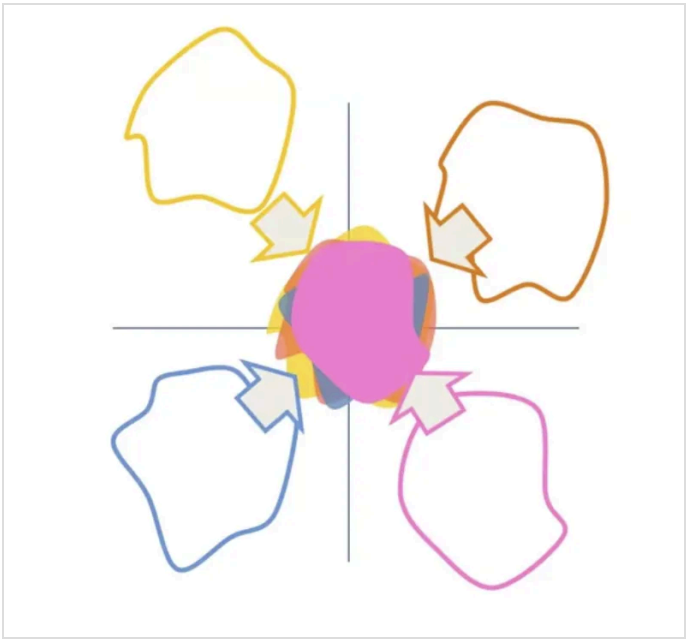
- 각 minibatch가 서로 너무 많은 정보를 담고있을 수 있음
- 모든 minibatch가 모두 다른 곳에 있을 수도 있음

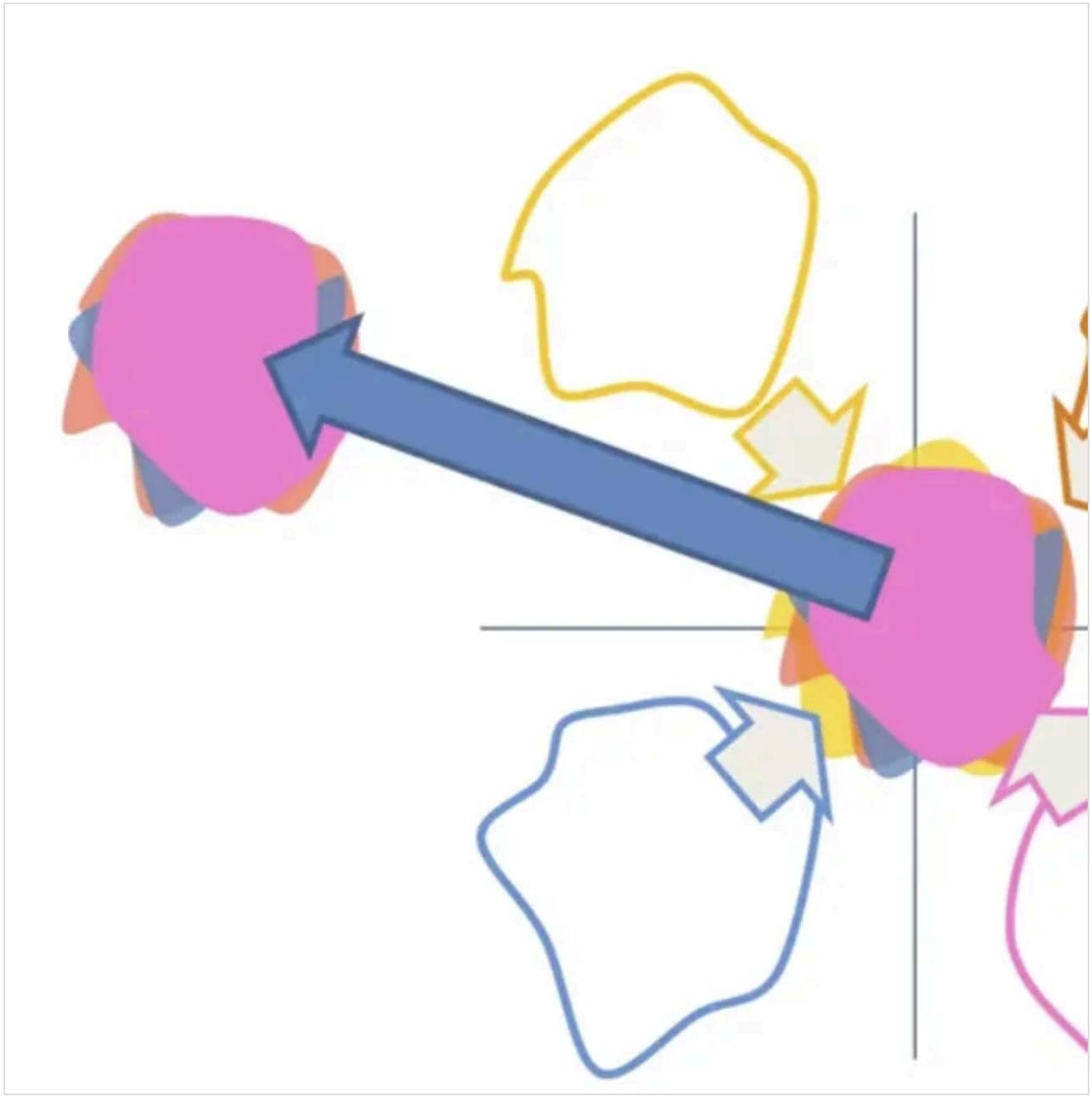


- → 모두 **표준 위치**로 보낼 수 있어야 함

Move all minibatch to a “standard” location

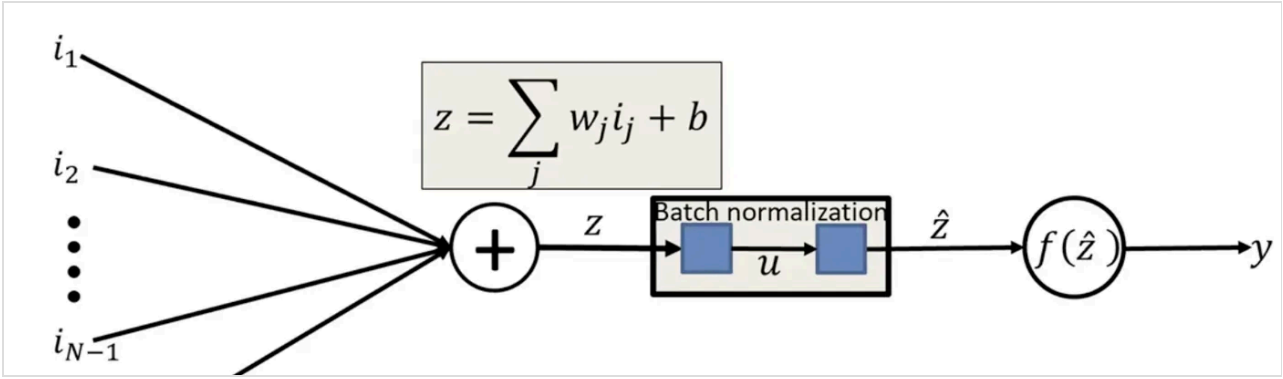


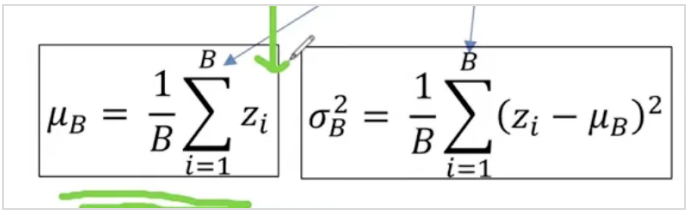


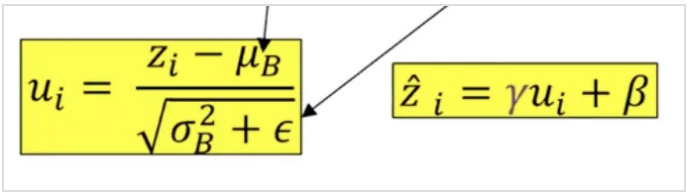


- 1. 원점으로 이동
- 2. 표준편차로 나눠서 분산을 같게 함
- 3. 고정된 위치로 이동

Batch normalization : Training

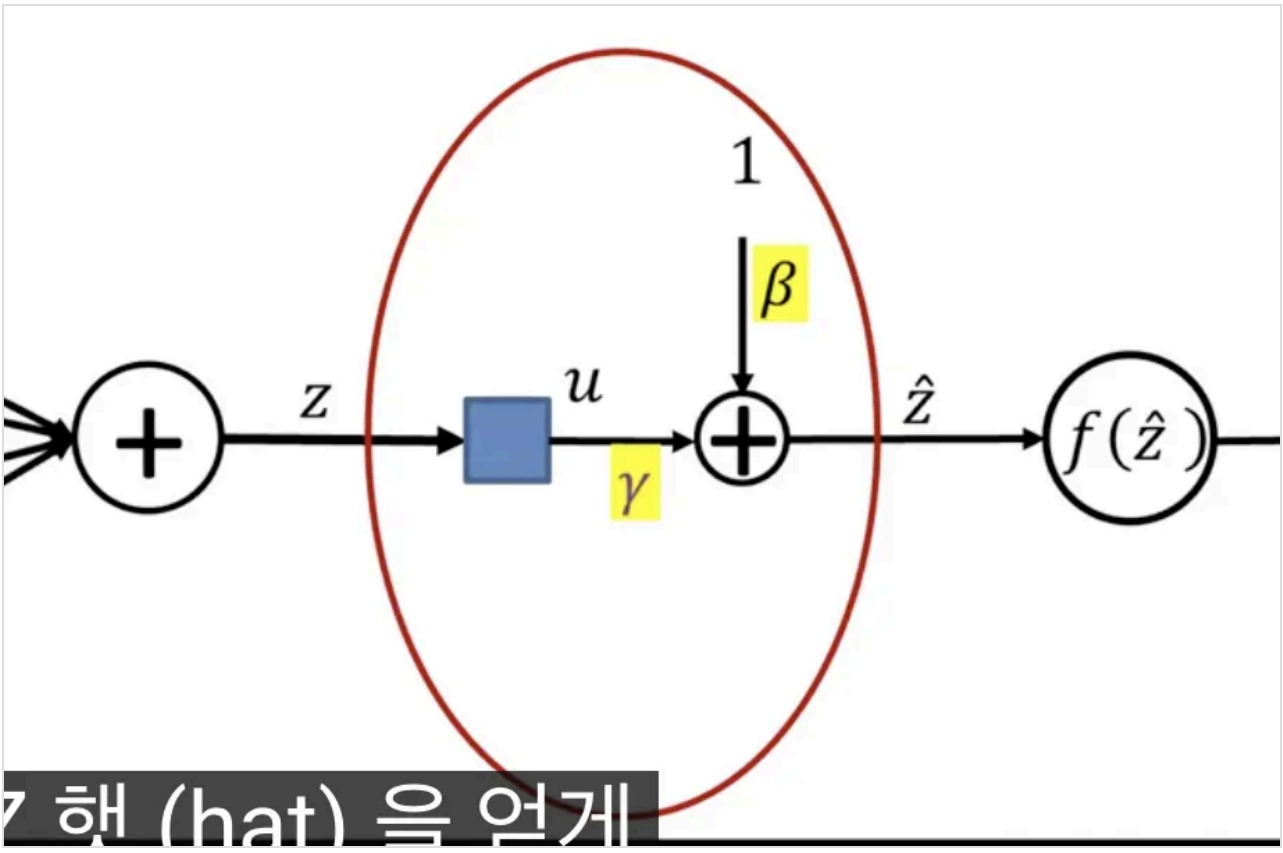






- B: minibatch size
- μ_B : Minibatch mean
  σ : Minibatch standard deviation

- 평균을 빼고,
  표준편차로 나눔.
- ⇒ **"Move all minibatch to a "standard" location" 부분이랑 같은 것을 하는 거임. (위에)**



Batch Normalization : Scale & Shift

- $\hat{z}_i = \gamma u_i + \beta$
  - **γ (Gamma)** : Scale → 분포의 크기 조절
    β (Beta) : Shift → 분포의 위치 이동
  - 정규화하면 평균 0, 분산 약 1로 제한됨.
    → γ와 β를 **학습 가능한 파라미터**로 설정하여 최적의 분포를 찾음.
- ⇒ **Batch Normalization = 정규화 + Scale & Shift**

Batch Normalization : Gradient 계산

$$Loss(minibatch) = \frac{1}{B}\sum_t Div(Y_t(X_t, \mu_B, \sigma_B^2), d_t(X_t))$$

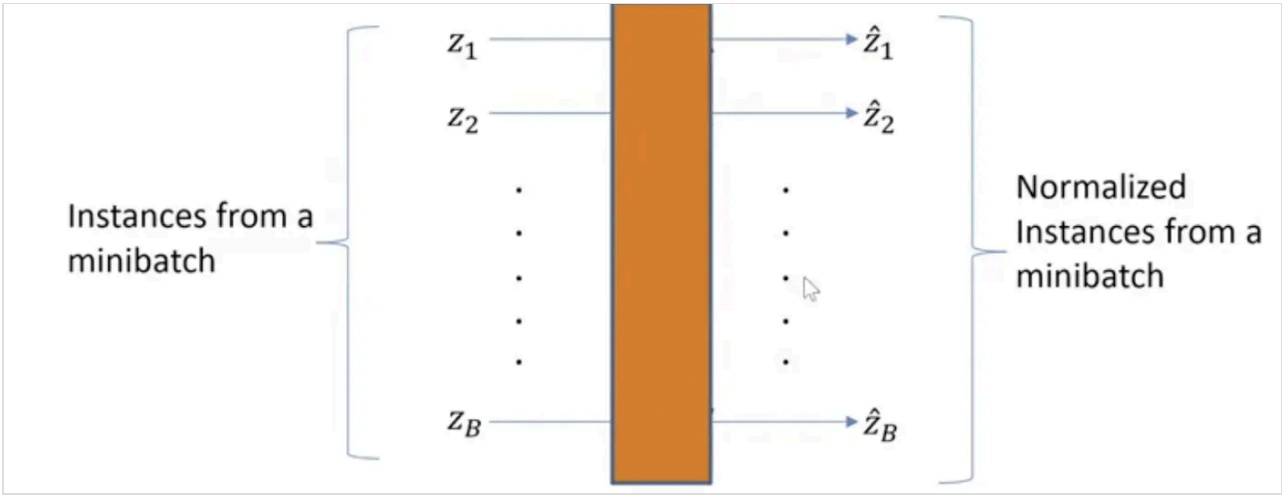
1. Without BN

- Loss = (1/B) Σ_i L_i
  ∇Loss = (1/B) Σ_i ∇L_i
  - → 각 데이터의 출력과 Gradient는 다른 데이터와 독립적임.

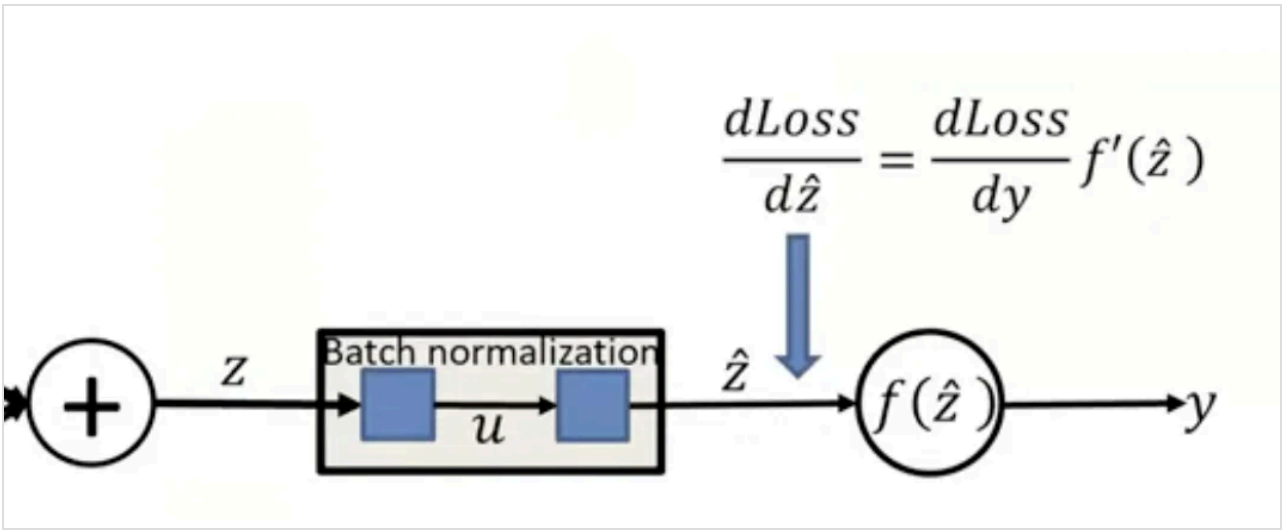
2. With BN(BatchNorm)

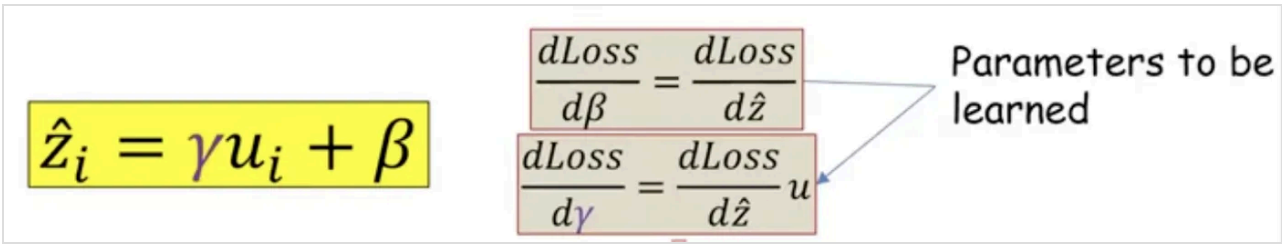
- BN은 Mini-batch 전체의 평균(μ_B)과 분산($\sigma^2{}_B$)을 사용함.
- $Loss = \frac{1}{B}\Sigma_i \; Div(Y_i \; (X_i \; , \mu_B, \sigma^2{}_B), d_i \;)$
- → $\mu_B, \sigma^2{}_B$가 모든 데이터에 의해 결정됨.
- → 각 데이터의 출력과 Loss가 **Batch 내 다른 데이터에도 의존**하게 됨.
- → Backpropagation 시 다른 데이터의 영향까지 고려해야 함.
- ⇒ **BN은 Mini-batch 내 데이터 간 의존성을 만들어 Gradient 계산을 복잡하게 함.**

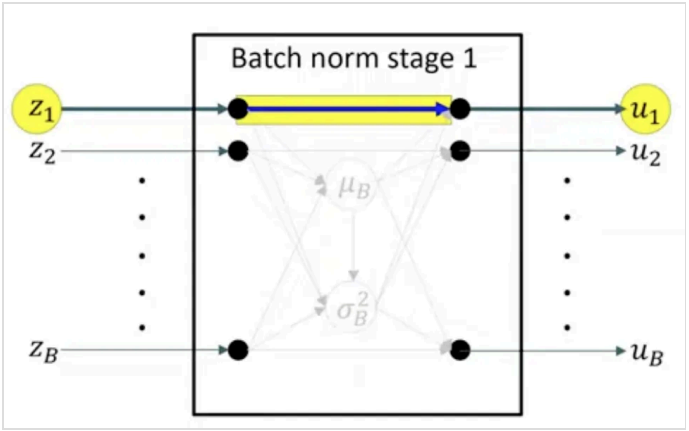
Batchnorm is a vector function over the minibatch

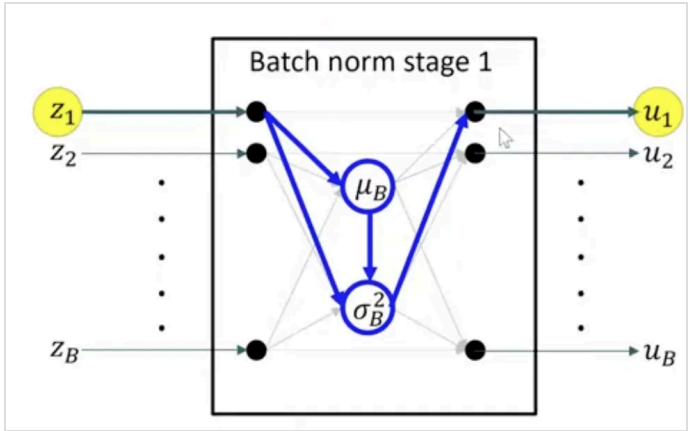


- Batch Normalization 는 vector function과 다를 게 없음.
  - → 왜냐하면, 모든 z가 $\hat{z}$에 영향을 주게 됨









- 각 z에 대해서, μ, σ 모두 바뀌며, 이에 의해 u가 바뀜.

The derivative for the "through" line ($i = j$)

$$\frac{du_i}{dz_i} = \frac{\partial u_i}{\partial z_i} + \frac{\partial u_i}{\partial \mu_B}\frac{\partial \mu_B}{\partial z_i} + \frac{\partial u_i}{\partial \sigma_B^2}\frac{d\sigma_B^2}{dz_i}$$

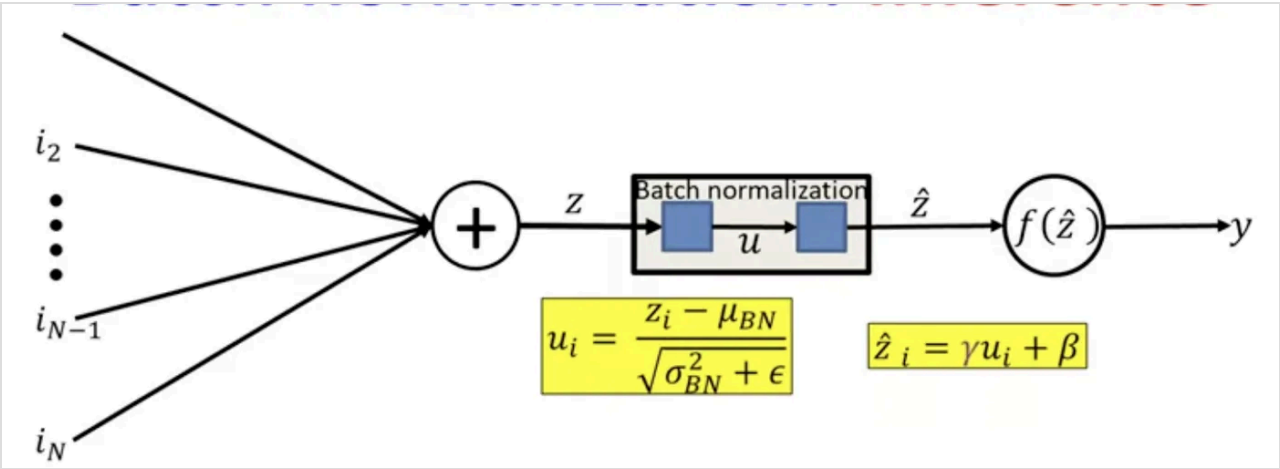
- 따라서, 이와 같이 $\frac{du_i}{dz_i}$가 되게 됨.

$$\frac{du_i}{dz_i} = \frac{1}{\sqrt{\sigma_B^2 + \epsilon}} + \frac{-1}{B\sqrt{\sigma_B^2 + \epsilon}} + \frac{-(z_i - \mu_B)}{2(\sigma_B^2 + \epsilon)^{3/2}}\frac{2(z_i - \mu_B)}{B}$$

- 각 미분결과를 넣으면 → 위와 같은 방정식을 구할 수 있음.

$$\frac{du_j}{dz_i} = \begin{cases} \frac{1}{\sqrt{\sigma_B^2 + \epsilon}} + \frac{-1}{B\sqrt{\sigma_B^2 + \epsilon}} + \frac{-(z_i - \mu_B)^2}{B(\sigma_B^2 + \epsilon)^{3/2}} & \text{if } j = i \\ \frac{-1}{B\sqrt{\sigma_B^2 + \epsilon}} + \frac{-(z_i - \mu_B)(z_j - \mu_B)}{B(\sigma_B^2 + \epsilon)^{3/2}} & \text{if } j \neq i \end{cases}$$

Batch normalization : Inference



$$\mu_{BN} = \frac{1}{Nbatches} \sum_{batch} \mu_B(batch)$$

$$\sigma_{BN}^2 = \frac{B}{(B-1)Nbatches} \sum_{batch} \sigma_B^2(batch)$$

Training

- 현재 Minibatch의 평균 μ_B, 분산 σ_B^2 사용.
  Minibatch가 바뀌면 정규화 기준도 달라짐.

Inference

- 새로운 데이터의 평균과 분산을 계산하지 않음.
  Training 중 누적한 평균 μ_{BN}, 분산 σ_{BN}^2를 사용.
  학습된 γ, β는 그대로 사용.

$$\hat{z}_i = \gamma \frac{z_i - \mu_{BN}}{\sqrt{\sigma_{BN}^2 + \epsilon}} + \beta$$

- **⇒ Training에서는 Minibatch별 통계량을 사용하지만, Inference에서는 학습 중 저장한 통계량을 고정해서 사용함.**

- ⇒ 따라서 Inference에서는 **다른 데이터에 의존하지 않고 개별 데이터를 예측할 수 있음.**

Batch Normalization : Application & Benefits

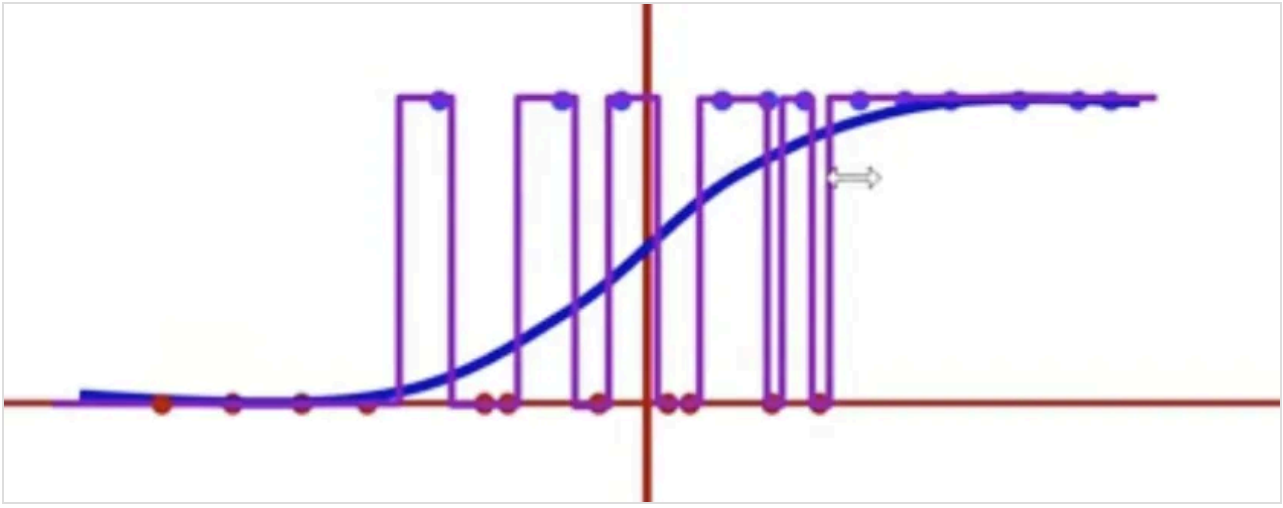
1. 적용 위치

- 일반적으로 Wx + b → BN → Activation 순서
  모든 Layer에 적용할 필요는 없음

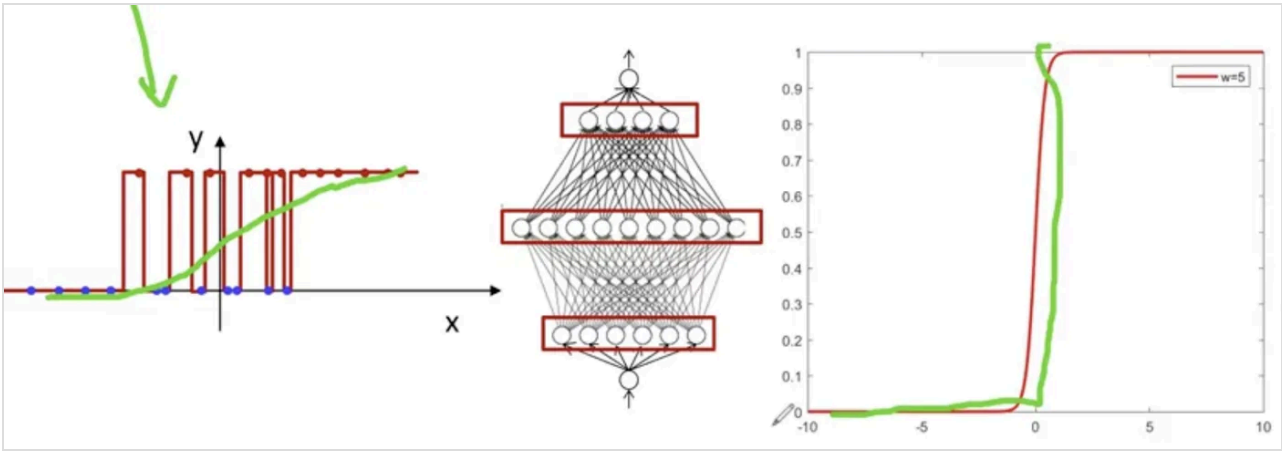
2. 효과

- **Faster Convergence : 학습 안정화 → 더 큰 Learning Rate 사용 가능**
  Better Generalization : Regularization 효과로 성능 개선 가능
  Less Dropout : BN이 일부 Regularization 역할 수행
  Faster LR Decay : Learning Rate를 더 빠르게 감소시킬 수 있음
  Data Shuffling : Minibatch 구성이 중요하므로 데이터 순서를 섞어야 함
- ⇒ **BN은 학습을 안정화하고 수렴 속도와 일반화 성능을 개선하는 데 도움을 줌.**

Overfitting and Smoothing



- 정답: 파란색 함수
  이 상황에서, 보라색으로 학습되는 것을 막기 힘듦.
- 방지하려면? → 왜 이게 학습되는지를 봐야 함.



- W가 커질수록, 더 가파라지면서, 이런 극단적인 경우를 학습하게 되는 거임.
- ⇒ W에 제한을 두면, 이런 overfitting을 막을 수 있음

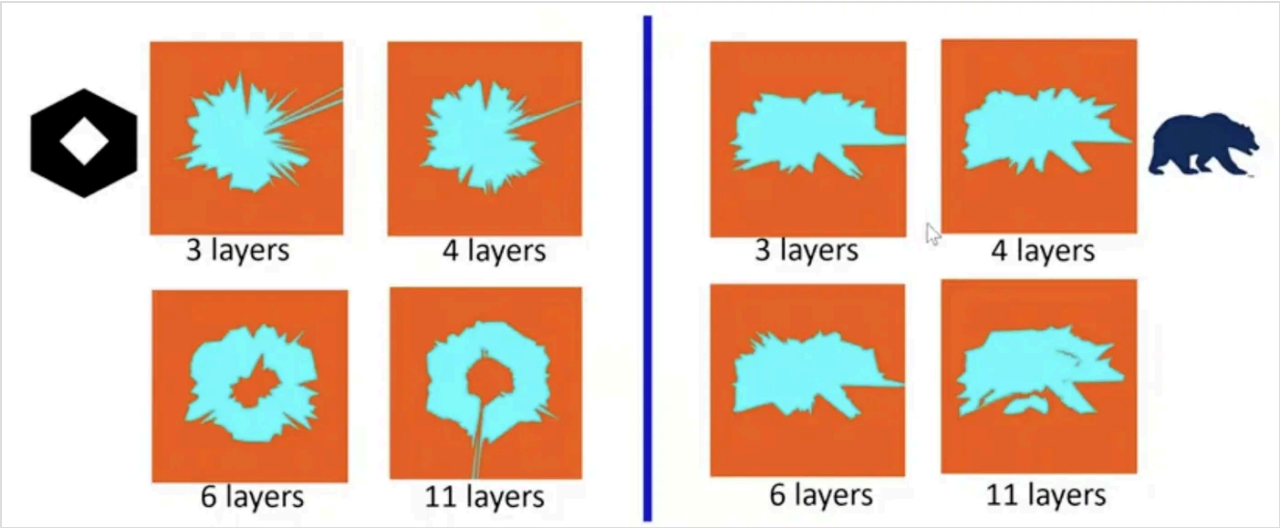
$$L(W_1, W_2, \dots, W_K) = \frac{1}{T}\sum_t Div(Y_t, d_t; W_1, W_2, \dots, W_K) + \frac{1}{2}\lambda \sum_k \|W_k\|_F^2$$

- = 정규화항을 두면 됨.

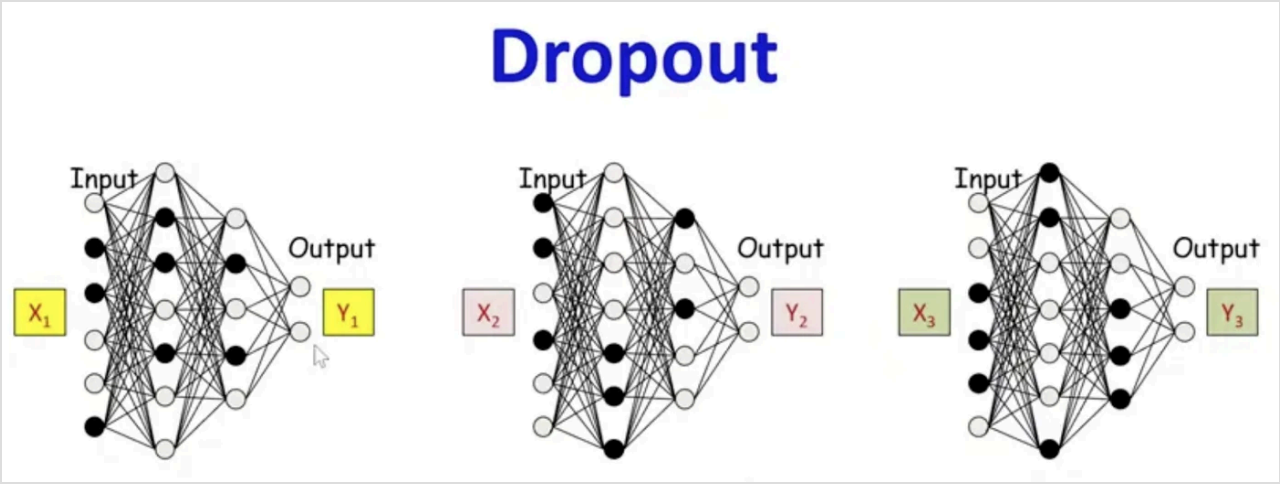
$$W_k \leftarrow W_k - \eta \nabla_{W_k} L^T = W_k - \eta \frac{1}{b} \sum \nabla_{W_k} Div(Y_t, d_t)^T - \eta\lambda W_k$$

$$W_k \leftarrow (1-\beta) W_k - \eta \frac{1}{b} \sum_{\tau=t}^{t+b-1} \nabla_{W_k} Div(Y_t, d_t)$$

- update할 때, W_k앞에 (1 - β)가 생기게 됨. (원래는 그냥 β)

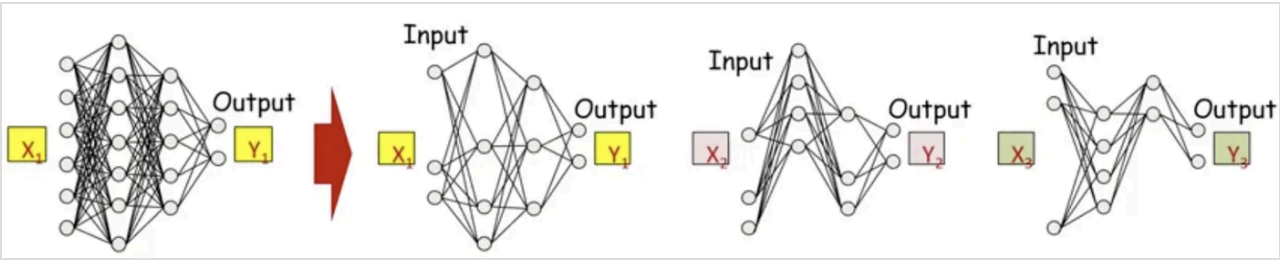


- depth → 더 정확한 학습을 할 수 있음(같은 뉴런 개수에서)

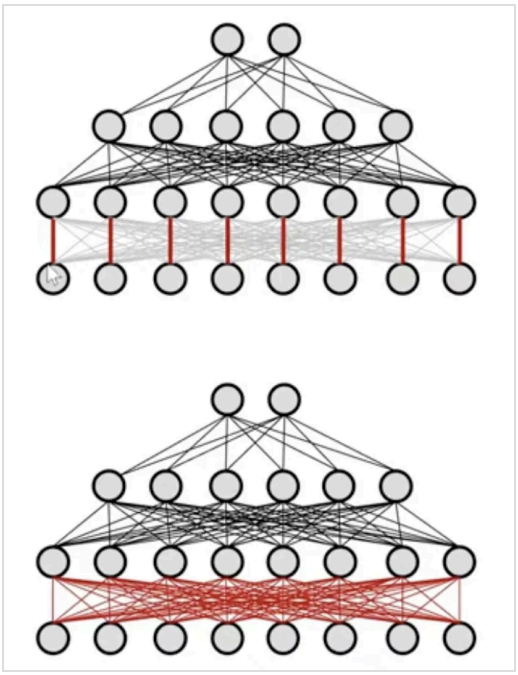


> **Dropout** : 학습하면서, 뉴런을 일부러 비활성화 함.

- training을 하면서,



- 필요하지 않은 node, edge를 버리면서, 학습할 수 있음
  - → 네트워크 구조 자체를 학습하게 되는 것.



Input: D dimensional vector $\mathbf{x} = [x_j,\ j = 1 \ldots D]$

Set:

- $D_0 = D$, is the width of the 0th (input) layer
- $y_j^{(0)} = x_j,\ j = 1 \ldots D; \quad y_0^{(k=1\ldots N)} = x_0 = 1$

For layer $k = 1 \ldots N$

Mask takes value 1 with prob. α, 0 with prob $1 - \alpha$

- $mask(k-1, j) = Bernoulli(\alpha)\ ,\ j = 1 \ldots D_{k-1}$
- $y_j^{(k-1)} = y_j^{(k-1)}.mask(k-1, j),\ j = 1 \ldots D_{k-1}$
- For $j = 1 \ldots D_k$
  - $z_j^{(k)} = \sum_{i=0}^{N_k} w_{i,j}^{(k)} y_i^{(k-1)} + b_j^{(k)}$
  - $y_j^{(k)} = f_k\left(z_j^{(k)}\right)$

Output:

- $Y = y_j^{(N)}, j = 1..D_N$

> **NN**은 이전의 레이어를 그대로 클론하는 경향이 있음

- **Drop Out**을 통해서, 이런 경향을 해결할 수 있음.

Training

- 뉴런을 무작위로 비활성화
  → N개 뉴런에 대해 최대 2^N개의 Sub-network를 학습하는 효과

Inference

- 이상적으로는 모든 Sub-network의 예측을 평균내는 Ensemble 사용
  → 계산량이 너무 많으므로 Expectation으로 근사
  - $E[f(y)] \approx f(E[y])$
  - → 모든 모델의 출력 평균 대신, 뉴런 출력의 기댓값을 사용하여 한 번만 계산

실제 구현

- 일반적인 Dropout : Inference에서 뉴런 출력을 유지 확률 p만큼 조정
  Inverted Dropout : Training에서 미리 보정하므로 Inference에서는 모든 뉴런을 그대로 사용
  ⇒ **Dropout은 여러 Sub-network의 Ensemble 효과를 하나의 네트워크로 근사하는 방법임.**

What each neuron computes

$$y_i^{(k)} = D\sigma\left(\sum_j w_{ji}^{(k)} y_j^{(k-1)} + b_i^{(k)}\right)$$

$$\mathrm{E}[y_i^{(k)}] = \alpha\sigma\left(\sum_j w_{ji}^{(k)} y_j^{(k-1)} + b_i^{(k)}\right)$$

- α : 뉴런이 살아남을 확률 (0\~1 사이의 고정값)
  $D \sim Bernoulli(\alpha)$: 확률 α로 1, 확률 $1-\alpha$로 0
  - D ⇒ (실제로 뉴런을 껐다킴)
- Training에서는 D를 곱해 뉴런을 무작위로 활성화/비활성화함.
  - → 뉴런 출력의 기댓값은 $E[y] = \alpha\sigma(z)$
    → **Inference에서는 Dropout을 끄고,**
    출력에 α를 곱해 기댓값을 사용함.
- ⇒ 여러 Sub-network의 Ensemble 효과를 근사하는 것.

$$W_{test} = \alpha W_{trained}$$